

ROS2’de A^* ve DWA Kullanarak TurtleBot3 iin Yol Planlama ve Dinamik Engelden Kaınma*

Yahye Abdır Rahman Bashır
Bursa Teknik niversitesi
Mekatronik Mhendislięi Blm
Mobil Robotlar ve Uygulamaları
yaxy2200@gmail.com

January 14, 2026

Abstract

Bu alıřma, ROS2 (Robot Operating System 2) atısını kullanarak karmařık ortamlarda otonom mobil robot navigasyonu problemini ele almaktadır. Proje, optimal yol retimi iin kresel planlayıcı (global planner) olarak **A^* (A-Star)** algoritmasını ve gerek zamanlı engelden kaınma iin yerel planlayıcı (local planner) olarak **Dynamic Window Approach (DWA)** algoritmasını birleřtiren saęlam bir yol planlama mimarisi uygulamaktadır. Sistem, Gazebo simlasyon ortamında bir TurtleBot3 Waffle modeli kullanılarak doęrulanmıřtır. zellikle, robotun navigasyon sırasında aniden ortaya ıkan dinamik engellere tepki verme yeteneęi test edilmiřtir. Sonular, nerilen Nav2 konfigrasyonunun kresel optimizasyonu ve yerel reaktiviteyi bařarılı bir řekilde yneterek robotun arpıřmadan hedefe ulařmasını saęladığını gstermektedir.

1 Giriř

Otonom navigasyon, bir mobil robotun evresini algılamasını, konumunu belirlemesini ve bir hedefe gvenli bir yol planlamasını gerektirir. Yol planlama tipik olarak iki katmana ayrılır: statik bir haritaya dayalı olarak yol bulan kresel planlama (global planning) ve dinamik engellerden kaınırken bu yolu takip etmek iin hız komutları reten yerel planlama (local

*Proje kaynak kodlarına řuradan eriřilebilir: https://github.com/yayabash/turtlebot3_path_planning

planning). İdeal olarak, küresel planlayıcı optimallliği sağlarken, yerel planlayıcı güvenliği ve kinematik uygulanabilirliği sağlar.

Bu projede, **ROS2 Humble** ara katman yazılımı (middleware) ve **Nav2** yığını (stack) kullanılmaktadır. Küresel planlama için *SmacPlanner* eklentisi aracılığıyla A* algoritmasını ve yerel kontrol için *DWBLocalPlanner* eklentisi aracılığıyla DWA algoritmasını uygulamaktayız. Bu çalışmanın önemli bir katkısı, dinamik "aniden beliren" (pop-up) engelleri ele almak için bu algoritmaların simüle edilmiş bir ortamda pratik uygulaması ve ayarlanmasıdır (tuning).

2 Literatür Taraması: A* ve Yol Planlama

A* algoritması, 1968'den beri mobil robotiğin temel taşlarından biri olmuştur. Güncel araştırmalar, dinamik ortamlarda performansını artırmaya ve hesaplama maliyetini düşürmeye odaklanmaktadır.

Zhang ve ark. (2025), A* algoritmasını Derin Q-Ağları (DQN) ve DWA ile birleştirmiştir. Hibrit yaklaşımları, robotların karmaşık ortamlarda yerel minimumlardan çıkmasını sağlarken, A*'ın küresel garantisini korumakta ve dinamik tepkiyi iyileştirmektedir [1].

Qin ve ark. (2025), Genetik Algoritmaları (GA) entegre ederek A*'ın arama uzayını optimize eden ve büyük ızgara haritalarında hesaplama yükünü önemli ölçüde azaltan "GO-GASA" yöntemini önermiştir [2].

Fu ve ark. (2025), engel yoğunluklu ortamlarda arama süresini kısaltan, optimize edilmiş bir sezgisel (heuristic) fonksiyona sahip "Geliştirilmiş A*" (Improved A*) yöntemini geliştirmiştir [3].

Duan ve ark. (2025), düzensiz haritalar için A*'ın ızgara tabanlı yapısını RRT*'ın (Rapidly-exploring Random Tree) örnekleme tabanlı doğasıyla birleştiren hibrit bir yöntem olan VGA*-RRT*'ı tanıtmışlardır [4].

Bu çalışmalar, A* temel olmaya devam ederken, bu uygulamanın odak noktası olan gerçek dünya uygulanabilirliği için DWA gibi reaktif yerel planlayıcılarla entegrasyonunun gerekli olduğunu vurgulamaktadır.

3 Metodoloji

Bu proje, diferansiyel sürüslü bir mobil robotu (TurtleBot3 Waffle) özel bir labirent benzeri ortamda simüle etmektedir.

3.1 Sistem Mimarisi (ROS2 Nav2)

Navigasyon yığını, iki temel maliyet haritası (costmap) ile yapılandırılmıştır:

1. **Global Costmap:** Statik bir doluluk ızgara haritası (*custom_map.yaml*) ve bir şişirme katmanı (inflation layer) kullanır. A* planlayıcısı burada çalışır.
2. **Local Costmap:** LIDAR sensörü tarafından 5 Hz'de güncellenen bir kayan pencere ($3m \times 3m$) kullanır. DWA kontrolcüsü yeni engellerden kaçınmak için burada çalışır.

3.2 Küresel Planlayıcı: A* (SmacPlanner)

A* algoritmasını kullanmak üzere yapılandırılmış `nav2_smac_planner/SmacPlanner2D` eklentisi seçilmiştir. A*, $f(n) = g(n) + h(n)$ maliyet fonksiyonunu minimize eder; burada $h(n)$ Öklid mesafesi sezgiselidir. Bu, bilinen statik haritada en kısa yolun bulunmasını sağlar.

3.3 Yerel Planlayıcı: DWA (DWB Controller)

Küresel yolu takip etmek ve çarpışmaları önlemek için `dwb_core::DWBLocalPlanner` özelleştirilmiştir. DWA algoritması, bir sonraki zaman adımında ulaşılabilir bir hız uzayını (v, ω) örnekler ve yörüngeleri şunlara göre değerlendirir:

- **Path Align (Yol Hizalama):** Küresel yol ile hizalanma.
- **Goal Align (Hedef Hizalama):** Hedefe doğru yönelim.
- **Base Obstacle (Temel Engel):** Engellere olan mesafe (statik ve dinamik).

Parametre Ayarı (Tuning): Robotun hedefte kendi etrafında dönmesi sorununu çözmek için `min_theta_velocity_threshold` değeri 0.01 rad/s olarak ayarlanmıştır. TF tampon (buffer) hatalarını önlemek için tüm düğümlerde `use_sim_time: true` ayarlanarak zaman senkronizasyonu zorunlu kılınmıştır.

4 Simülasyon ve Test

4.1 Ortam Kurulumu

Simülasyon **Gazebo**'da barındırılmaktadır. Çalışma alanı yapısı şunları içerir:

- `turtlebot3_gazebo/launch/turtlebot3_my_world.launch.py`: Ortamı başlatır.
- `turtlebot3_navigation2/param/waffle_astar.yaml`: Ayarlanmış Nav2 parametrelerini içerir.

4.2 Dinamik Engel Deneyi

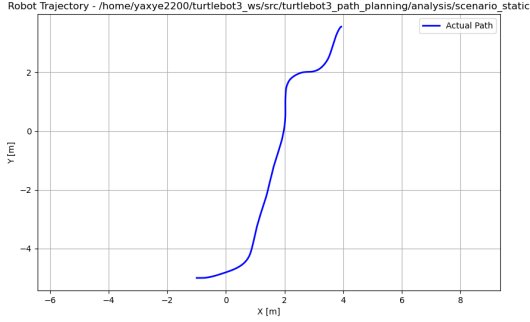
Robot hareket halindeyken belirtilen koordinatlarda (x, y) geometrik bir engel (kırmızı bir kutu) oluşturmak (spawn) için özel bir Python betiği olan `spawn_box.py` geliştirilmiştir.

Test Prosedürü:

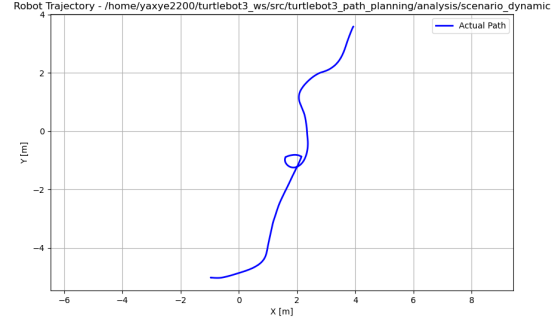
1. Simülasyonu ve Navigasyon yığını başlatın.
2. RViz'de bir 2D Navigasyon Hedefi belirleyin.
3. Robot hareket ederken şu komutu çalıştırın: `python3 spawn_box.py`.
4. Yerel Costmap güncellemesini ve DWA kontrolcüsünün yörüngeyi ayarlamasını gözlemleyin.

5 Sonuçlar

Önerilen navigasyon mimarisinin performansı, hem yerel kaçınma hem de küresel yeniden planlama yeteneklerini test etmek için iki farklı dinamik senaryoda değerlendirilmiştir. Her senaryo için, "Statik" bir temel koşu, işlemin ortasında bir engelin oluşturulduğu "Dinamik" bir koşu ile karşılaştırılmıştır.



(a) Statik ortam



(b) Küçük dinamik engel

Figure 1: Statik navigasyon ve dinamik küçük engelden kaçınma arasındaki karşılaştırma.

5.1 Senaryo A: Küçük Engelden Kaçınma

Bu deneyde, robotun oluşturulan yolu üzerine küçük bir engel yerleştirilmiştir. DWA yerel planlayıcısı, robotu engelin etrafından geçirmek için başarılı bir şekilde devreye girmiştir.

Table 1: Performans Metrikleri: Küçük Engel Senaryosu

Test Koşulu	Toplam Mesafe (m)	Toplam Süre (s)
Statik Temel (Baseline)	10.97	56.56
Dinamik Engel	12.78	67.34

Tablo 1’te detaylandırıldığı gibi, etkileşim seyahat süresinde orta düzeyde bir artışa ($\approx 15\%$) yol açmıştır. Özellikle, kat edilen toplam mesafe önemli ölçüde artmıştır (11.01m’den 12.78m’ye). Bu durum, yerel planlayıcının engeli güvenli bir şekilde baypas etmek için yol optimallliği yerine güvenliği önceliklendirerek geniş ve korumacı bir yörünge ürettiğini göstermektedir.

5.2 Senaryo B: Yol Tıkanıklığı (Büyük Engel)

İkinci deneyde, optimal A* yolunu tamamen tıkayan büyük bir engel oluşturulmuştur. Bu durum, yerel planlayıcının başarısız olmasına, kurtarma davranışlarını (durma) tetiklemesine ve ardından küresel bir yeniden planlama olayına yol açmıştır.

Tablo 2, ilk senaryoya kıyasla bir tezatlığı ortaya koymaktadır. Seyahat mesafesi nispeten stabil kalmıştır (+9m), ancak toplam seyahat süresi neredeyse iki katına çıkmıştır

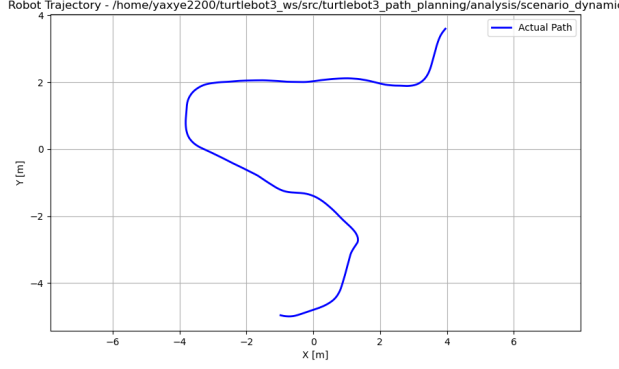


Table 2: Performans Metrikleri: Yol Tıkanıklığı Senaryosu

Test Koşulu	Mes. (m)	Süre (s)
Statik Temel	10.97	56.99
Dinamik Engel	20.11	111.01

Figure 2: Orijinal A* yolunu tıkayan büyük dinamik engelle tetiklenen küresel yeniden planlama.

(56.99s'den 111.01s'ye). Bu önemli gecikme, robotun pratik olarak hareketsiz kaldığı süreye atfedilmektedir: tıkanıklığı algılama, maliyet haritasının güncellenmesini bekleme ve harekete devam etmeden önce blokajın etrafından yeni bir küresel yol hesaplama.

6 Tartışma

İki deneysel senaryonun karşılaştırmalı analizi, ROS2 Nav2'deki yerel ve küresel planlama katmanlarının farklı rollerini vurgulamaktadır.

6.1 Yerel Kaçınma ve Küresel Yeniden Planlama Karşılaştırması

- **Verimlilik Takası (Trade-off):** Senaryo A'da (Küçük Engel), sistem *zamansal verimliliği* tercih etmiştir. Yerel planlayıcı (DWA), robotun momentumunu koruyarak durmadan engelden kaçınmıştır. Ancak bu, yol uzunluğundaki %15'lik artışta (12.78m vs 11.01m) görüldüğü gibi *mekansal verimlilik* pahasına gerçekleşmiştir. Yerel planlayıcı "açgözlüdür" (greedy) ve engeli derhal aşmak için mekansal olarak optimal olmayan geniş bir sapma yolu seçebilir.
- **Yeniden Planlama Gecikmesi:** Senaryo B'de (Büyük Engel), sistem *yol uygulanabilirliğini* önceliklendirmiştir. Rota tıkandığı için genel yerel planlayıcı ilerleyememiştir. Sonraki küresel yeniden planlama, mekansal olarak optimal yeni bir yolun bulunmasını sağlamış (sadece %48 mesafe artışı), ancak bu ciddi bir zaman cezasına (%55 artış) neden olmuştur. Bu gecikme, "kurtarma davranışı" yaşam döngüsünü açıklar: durma → maliyet haritalarını temizleme → A* hesaplama → kontrole devam etme.

6.2 Gerçek Dünya Dağıtımı İçin Çıkarımlar

Bu sonuçlar, küçük ve hareketli varlıkların (örn. insanlar) bulunduğu dinamik ortamlar için iyi ayarlanmış bir yerel planlayıcının yeterli olduğunu ve sorunsuz operasyonu sürdürmek için tercih edildiğini göstermektedir. Ancak, yapısal değişiklikler (örn. kapalı kapılar, inşaat) için sistem, küresel yeniden planlama için gereken zaman gecikmesini kabul edecek

kadar sağlam olmalıdır. Senaryo A'daki önemli mesafe sapması ayrıca, eğer enerji tasarrufu (kat edilen mesafe) hızdan daha yüksek bir öncelikse, engellerin etrafında daha dar dönüşleri teşvik etmek için DWA `PathDist` değerlendiricisinin (critic) daha fazla ayarlanması gerekebileceğini düşündürmektedir.

7 Sonuç

Bu proje, sağlam mobil robot navigasyonu için ROS2'de A* ve DWA'nın birleştirilmesinin etkinliğini göstermiştir. A* algoritması güvenilir bir küresel rota sağlarken, ayarlanmış DWA kontrolcüsü kinematik kısıtlamaları ve dinamik engelleri ele almıştır. Simülasyon zaman senkronizasyonu ve hedef toleransı ile ilgili sorunlar tanımlanmış ve parametre optimizasyonu ile çözülmüştür.

References

- [1] Y. Zhang, C. Cui, and Q. Zhao, "Path planning of mobile robot based on a star algorithm combining dqn and dwa in complex environment," *Applied Sciences*, vol. 15, no. 8, p. 4367, 2025.
- [2] J. Qin et al., "Global path planning based on go-gasa algorithm," *Robotics and Autonomous Systems*, 2025.
- [3] X. Fu, Z. Huang, and J. Wang, "Research on path planning of mobile robots based on improved a* algorithm," *PeerJ Computer Science*, vol. 11, e2691, 2025.
- [4] L. Duan et al., "Vga*-rrt*: A hybrid path planning algorithm," *Journal of Field Robotics*, 2025.

Project GitHub Repository:

https://github.com/yayabash/turtlebot3_path_planning
